

Skan^{AI}

AGENTIC ONTOLOGY OF WORK (AOW)

A foundational semantic model for
intelligent, autonomous, and governed
enterprise workflows

Table of Contents

- 1 Introduction 2
- 2 Conceptual Landscape of the Agentic Enterprise 6
- 3 Canonical Entities of the AOW 9
- 4 Ontological Relationship Model 17
- 5 The AOW Lifecycle Model 21
- 6 Metadata Schemas 24
- 7 Example: Claims Processing Scenario 27
- 8 Strategic Significance of AOW 29
- 9 How AOW Relates to Existing Frameworks 30
- 10 Industry Example Set 33
- 11 Validation Criteria 35
- 12 Glossary 37
- Appendix A: Formal Relationship Matrix 39
- Appendix B: Example Ontology Registry Format 41
- 13 Summary 42

1 Introduction

1.1 The Rise of the Agentic Enterprise

The modern enterprise is undergoing a foundational shift—from systems that *automate tasks* to systems that *understand, reason, and act* across complex operational landscapes. The convergence of drives this transformation:

- multi-agent systems
- learned behaviors
- contextual awareness
- structured governance
- continuous feedback loops
- enterprise-scale observability

Yet while the technological capability to build agentic systems is emerging rapidly, enterprises lack a **shared language** to describe what these systems are, how they behave, how they interrelate, and how they must be governed.

Just as **Service-Oriented Architecture (SOA)** provided the first widely adopted formalism to describe services, endpoints, and contracts, the Agentic era requires a **new ontology**—one that describes **Intents, Agents, Skills, Policies, Contexts, Outcomes, Assurance, Memory, Feedback**, and more.

1.2 Why an Ontology Is Essential

An ontology is not merely a glossary; it is a **formal semantic model** describing:

- the entities relevant to a domain,
- the attributes of those entities, and
- the relationships that connect them.

Without a shared ontology, enterprises face:

- inconsistent definitions (e.g., "agent," "workflow," "task" overlap)
- brittle integrations
- governance blind spots
- ambiguous responsibility models
- inability to scale autonomous activity

A well-designed ontology:

- anchors governance and compliance,
- enables explainability,
- improves cross-platform interoperability,
- accelerates enterprise adoption,
- reduces ambiguity in system behavior, and
- becomes the "grammar" of how intelligent work is described.

1.3 Goals of the Agentic Ontology of Work (AOW)

The AOW aims to:

- Establish a platform-agnostic semantic framework.
- Enable interoperability across multi-agent systems, RPA systems, AI systems, and human workflows.
- Support governance, auditability, and risk management.

- Provide a foundation for future standards bodies and reference architectures.
- Enable enterprises to scale intelligent automation with trust and transparency safely.

1.4 Design Principles

The AOW is built on six design principles:

1.5 Principle 1 — Platform Agnosticism

The ontology must describe concepts, not implementations. No vendor-specific models, training mechanisms, or product metaphors are embedded.

1.6 Principle 2 — Hierarchical Simplicity and Semantic Completeness

The ontology must be rich enough to model complex work, but simple enough to be usable by architects, ML engineers, and non-technical stakeholders.

1.7 Principle 3 — Governed Autonomy

Agentic systems operate under constraints, not unchecked freedom. Policies, Assurance Levels, and Provenance are first-class entities.

1.8 Principle 4 — Observability and Explainability

No action happens without visibility. No decision is made without a traceable rationale.

1.9 Principle 5 – Contextual Intelligence

Actions must always be understood in relation to environment, history, constraints, and Outcomes.

1.10 Principle 6 – Continuous Learning

The ontology must support experience accumulation and adaptation as environments shift and agents evolve.

2 Conceptual Landscape of the Agentic Enterprise

2.1 Work as a Semantic Graph

Work is not linear—it is a graph of relationships connecting:

- Objectives
- Intents
- Context
- Constraints
- Actors
- Actions
- Observations
- Feedback
- Outcomes

The AOW models this graph with entities and relationship types.

2.2 The Four-Layer Cognitive-Operational Stack

The ontology describes agentic systems across four layers:

2.3 Layer 1: Perception (Observation and Sensing)

Captures signals, including events, states, documents, logs, interactions, and structured/unstructured telemetry. This is the **ground truth** substrate.

Key conceptual elements:

- Telemetry Events
- Signals

- Session State
- Environmental Observations

2.3.1 Layer 2: Cognition (Understanding and Reasoning)

Evaluates goals, constraints, context, and policies to determine what should happen next.

Entities introduced:

- Objective
- Intent
- Context
- Policy
- Plan (optional intermediate concept)

2.3.2 Layer 3: Execution (Action and Collaboration)

Actions are performed by actors (Agents). Skills define what they can do. Results represent atomic effects.

Execution entities:

- Orchestrator
- Agent
- Skill
- Action
- Result

2.3.3 Layer 4: Assurance (Governance, Risk, Learning)

Ensures safety, reliability, trust, and improvement.

Assurance entities:

- Confidence
- Assurance Level
- Outcome
- Feedback
- Memory
- Guardian
- Provenance & Versioning

3 Canonical Entities of the AOW

3.1 Objective

3.1.1 Definition

A business-level goal expressed in strategic or operational terms.

3.1.2 Purpose

Connects agentic activity to specific enterprise value.

3.1.3 Attributes

- objective_id
- description
- kpi_target
- priority
- time_horizon

3.1.4 Relationships

- Produces → Intent(s)
- Evaluated through → Outcome(s)

3.2 Intent

3.2.1 Definition

A structured, actionable goal derived from an Objective and interpretable by agents or orchestrators.

3.2.2 Attributes

- intent_id
- goal_statement
- constraints
- priority
- acceptable_risk_band
- assurance_required

3.2.3 Key Distinction

Objective = why

Intent = what

3.2.4 Relationships

- Shaped by → Context
- Governed by → Policy
- Decomposed by → Orchestrator
- Fulfilled by → Agent(s)

3.3 Context

3.3.1 Definition

The structured situational information that gives relevance to an Intent.

3.3.2 Types of Context

- Environmental
- Procedural
- Data-level
- Historical

- Session-specific

3.3.3 Attributes

- context_id
- variables
- sensitivity_level
- provenance

3.3.4 Relationships

- Informs → Intent
- Retrieved from → Memory

3.4 Policy

3.4.1 Definition

Declarative constraints defining permissible or restricted behaviors.

3.4.2 Attributes

- policy_id
- rule_set
- trigger_conditions
- severity
- override_protocol

3.4.3 Relationships

- Constrains → Agent, Intent, Skill
- Enforced by → Guardian

3.5 Orchestrator

3.5.1 Definition

A coordinating entity that decomposes Intents into executable tasks, assigns them to Agents, and manages sequencing.

3.5.2 Attributes

- `planning_model`
- `dependency_graph`
- `exception_protocol`
- `resource_constraints`

3.5.3 Relationships

- Receives → Intent
- Delegates to → Agent
- Consults → Policy

3.6 Agent

3.6.1 Definition

A policy-aware actor capable of interpreting Intents and acting upon them using Skills.

3.6.2 Attributes

- `agent_id`
- `role`
- `skills`
- `autonomy_level`
- `assurance_level_required`
- `memory_link`

3.6.3 Relationships

- Invokes → Skills
- Produces → Results
- Writes → Memory
- Constrained by → Policy

3.7 Skill

3.7.1 Definition

A reusable capability callable by an Agent.

3.7.2 Attributes

- skill_id
- input_schema
- output_schema
- binding
- latency
- cost_profile

3.7.3 Relationships

- Used by → Agent
- Produces → Result

3.8 Result

3.8.1 Definition

The immediate effect or atomic output produced by executing a Skill.

3.8.2 Attributes

- result_id
- status
- artifacts
- confidence_measure

3.8.3 Relationships

- Aggregates into → Outcome

3.9 Outcome

3.9.1 Definition

The aggregated business impact from one or more Results.

3.9.2 Attributes

- outcome_id
- kpi_impact
- time_to_value
- risk_profile_change

3.9.3 Relationships

- Feeds → Feedback
- Evaluates → Objective

3.10 Confidence

3.10.1 Definition

A model- or rule-derived measure of certainty about a perception, decision, or action.

3.10.2 Attributes

- confidence_id
- value
- source (model heuristic, rule, ensemble)
- volatility

3.10.3 Relationships

- Compared to → Assurance Level

3.11 Assurance Level

3.11.1 Definition

A governance-required threshold determining autonomy vs. human oversight.

3.11.2 Attributes

- level
- review_protocol
- risk_class

3.12 Feedback

3.12.1 Definition

The structured information used to improve future behavior.

3.12.2 Attributes

- feedback_type
- issue_detected
- recommended_adjustments

3.12.3 Relationships

- Updates → Policy, Memory

3.13 Memory

3.13.1 Definition

Persistent storage of knowledge from experience.

3.13.2 Types

- 1. Operational Memory** — short-lived, execution-oriented
- 2. Knowledge Base** — long-term, generalized patterns

3.14 Guardian

3.14.1 Definition

The oversight mechanism that enforces governance, policy adherence, explainability, and auditability.

3.15 Provenance & Versioning

3.15.1 Definition

Metadata that captures the lineage, origin, and evolution of all entities.

4 Ontological Relationship Model

While the previous section detailed the core entities, the true value of the ontology emerges when we describe **how these entities relate**.

4.1 High-Level Semantic Relationships

At its core, the AOW is a **directed, constraint-aware, learning-enabled semantic graph** connecting business goals to execution behavior and systemic learning.

Below is a structured description of the relationships:

4.1.1 Objective → Intent

- An **Objective** spawns one or more **Intents**, each representing a specific, actionable component of the broader business goal.

4.1.2 Intent → Context

- Intents are shaped and constrained by **Context**, which provides environmental relevance and situational data.

4.1.3 Intent → Orchestrator

- The **Orchestrator** translates the Intent into an executable plan, decomposing it into tasks or steps.

4.1.4 Orchestrator → Agent

- The Orchestrator assigns tasks to **Agents**, chosen based on role, capability, autonomy level, and Assurance requirements.

4.1.5 Agent → Skill

- Agents invoke **Skills**, each representing an atomic capability.

4.1.6 Skill → Result

- Skills produce **Results**, immediate atomic effects that reflect the execution behavior.

4.1.7 Results → Outcome

- Multiple Results aggregate into **Outcomes**, metrics that map back to business KPIs and the originating Objective.

4.1.8 Outcome → Feedback

- Outcomes generate **Feedback**, which updates Policies, Memory, or Intent patterns.

4.1.9 Feedback → Memory

- Feedback flows into **Memory**, contributing to either:
 - **Operational Memory** (short-lived execution context), or
 - **Knowledge Base** (long-term patterns, rules, signals)

4.1.10 Memory ↔ Context

- Memory enriches future Context; updated Context produces more accurate Intent interpretation.

4.1.11 Policy ↔ Agent/Intent/Skill

- Policies constrain what Agents can do, what Intents can trigger, and what Skills are permissible under certain conditions.

4.1.12 Guardian → Everything

The **Guardian** enforces:

- Policy adherence
- Assurance Levels
- Risk thresholds
- Provenance correctness
- Auditability
- Override protocols

4.2 Semantic Graph of AOW

The following describes the semantic structure:

[Objective]

↓ produces

[Intent] ← shaped by ← [Context] ← informed by ← [Memory]

↓ decomposed by

[Orchestrator]

↓ assigns tasks to

[Agent] ← constrained by ← [Policy] ← enforced by ←

[Guardian]

↓ invokes

[Skill]

↓ yields

[Result] ← annotated with ← [Confidence] → compared against

→ [Assurance Level]

↓ aggregates into

[Outcome]

↓ generates

[Feedback]

↓ updates

[Memory] ← maintains ← [Provenance & Versioning]

5 The AOW Lifecycle Model

Now that the ontology defines entities and relationships, here is the **full lifecycle** of agentic execution from business goal to autonomous improvement.

5.1 Stage 1: Goal Setting (Business Layer)

- Business defines **Objectives**, often tied to KPIs, SLAs, regulatory requirements, or operational targets.
- Example Objective: Reduce claims processing time by 20%.

5.2 Stage 2: Intent Formation (Cognition Layer)

- Intent structures the Objective into machine-interpretable form.
- Example Intent: Automate initial claims validation.

5.3 Stage 3: Context Enrichment

- Real-time context (customer data, system state, environment, historical patterns) enriches the Intent.
- Example Context: Claim type = disability; claimant age = 45; policy rules = X.

5.4 Stage 4: Policy Application

- Policies filter what is permitted, required, or restricted.
- Example Policy: Auto-approve claims < \$50,000 if confidence ≥ 0.9 .

5.5 Stage 5: Orchestration Planning

- Orchestrator generates a task graph:
 - Task 1: Extract claim details
 - Task 2: Validate coverage
 - Task 3: Calculate benefit

5.6 Stage 6: Execution (Agent + Skill Invocation)

- Agents selected based on Skills, autonomy, and constraints.
- Skills are invoked atomically.

5.7 Stage 7: Result Generation

Each executed Skill produces a Result, annotated with:

- Confidence measure
- Relevant artifacts (documents, logs, state changes)
- Execution metadata

5.8 Stage 8: Assurance Verification

The **Guardian**:

- Validates compliance
- Checks Assurance Levels
- Verifies Policies
- Ensures Provenance correctness

5.9 Stage 9: Outcome Assessment

Results aggregate into Outcomes:

- Cycle time reduction
- Error rate changes

- SLA improvements

5.10 Stage 10: Feedback Loop

Feedback updates:

- Policies
- Knowledge Base
- Agent behavioral patterns
- Intent decomposition strategies

5.11 Stage 11: Memory Update

Execution is captured and stored for:

- Future context enrichment
- Knowledge improvement
- Model retraining
- Audit trails

6 Metadata Schemas

To support interoperability across platforms, the AOW defines **platform-agnostic metadata schemas** for each entity.

Below are illustrative examples in YAML (though JSON or RDF could be used).

6.1 Intent Schema

intent:

id: "intent-eligibility-2025"

objective_ref: "obj-claims-efficiency"

goal_statement: "Validate claim eligibility."

priority: "High"

constraints:

- "must comply with policy-auto-approval."

context_ref: "ctx-claim-84933"

assurance_required: "AL2"

6.2 Agent Schema

agent:

id: "eligibility-agent-v3"

role: "process-agent"

autonomy_level: "Delegated"

skills:

- "check_coverage"
- "calculate_benefit"

policy_refs:

- "policy-auto-approval"

memory_links:

operational: "mem-runtime-2025"

knowledge_base: "kb-claims-v1"

6.3 Skill Schema

skill:

id: "calculate_benefit"

input_schema:

claim_amount: "number"

policy_rules: "object"

output_schema:

benefit_amount: "number"

constraints:

- "return in < 1 second."

cost_profile: "low"

provenance:

created_by: "team-claims"

version: "1.4.2"

6.4 Feedback Schema

feedback:

id: "fbk-claim-584"

result_ref: "result-claim-584-step3"

type: "accuracy-adjustment"

recommended_adjustments:

- "increase confidence threshold to 0.92"

severity: "medium"

timestamp: "2025-04-10T10:22:41Z"

7 Example: Claims Processing Scenario

Here is a sample **semantic flow** using AOW:

7.1 Objective

- Reduce claims cycle time by 20%.

7.2 Intent

- Automate eligibility determination for disability claims.

7.3 Context

- Claimant demographics
- Policy rules
- Past claims patterns
- Regulatory constraints

7.4 Policies

- Auto-approve claims < \$50K with confidence ≥ 0.9
- Human review required for high-risk categories

7.5 Orchestrator Task Graph

1. Extract claim metadata
2. Validate coverage
3. Determine benefit
4. Generate decision

7.6 Agents

- Coverage-Agent
- Validation-Agent
- Benefit-Agent

7.7 Skills

- check_coverage()
- rule_validate()
- calculate_benefit()

7.8 Result

- Claim eligibility = Approved
- Confidence = 0.93

7.9 Assurance Level

- AL2 (delegated autonomy, optional human confirmation)

7.10 Outcome

- Turnaround reduced from 48 hours to 7 minutes

7.11 Feedback

- Add new rule exception for age > 65

7.12 Memory Update

- Append case trace to KB for future pattern learning

8 Strategic Significance of AOW

AOW provides enterprises with:

8.1 A Universal Language for Intelligent Work

Aligns business, engineering, governance, and automation teams.

8.2 Governance-First Automation

Trust and auditability built into every entity.

8.3 Cross-Platform Interoperability

RPA bots, LLMs, multi-agent systems, BPM engines, and human workflows can all interface via shared semantics.

8.4 Foundation for Standardization

Future industry consortiums or regulators could adopt AOW as a reference model.

8.5 Enterprise AI Readiness

Simplifies risk management, adoption, scaling, and integration.

9 How AOW Relates to Existing Frameworks

To fully establish AOW as a category-defining model, it is essential to understand how it aligns with, diverges from, and extends prior frameworks. This contextualization not only clarifies its value but also positions AOW for adoption by enterprise architects, AI governance teams, academics, and standards bodies.

9.1 AOW vs. SOA (Service-Oriented Architecture)

9.1.1 Commonalities

- Both define structured entities and formal contracts
- Both aim to standardize interoperability
- Both emphasize discoverability and governance

9.1.2 Differences

SOA Concept	AOW Equivalent	Key Difference
Service	Agent	Agents reason, plan, and act; services do not
Operation	Skill	Skills include execution metadata + governance

Service Contract	Policy / Assurance	AOW introduces confidence + risk
Orchestration	Orchestrator	AOW plans dynamically under context
UDDI Registry	Ontology Registry	AOW models learning and outcomes

AOW extends SOA from static invocation to contextual, adaptive execution.

9.2 AOW vs. BPMN (Business Process Modeling Notation)

9.2.1 Commonalities

- Both describe workflows and dependencies
- Both support cross-entity coordination

9.2.2 Differences

- BPMN assumes deterministic flows; AOW supports **dynamic, context-aware decisioning**
- BPMN models tasks, not reasoning; AOW models **objectives, context, policies, outcomes**
- BPMN is flow-first; AOW is **semantic-first**

AOW is not a process map — it is a cognitive representation of enterprise work.

9.3 AOW vs. Multi-Agent Systems (MAS) Research

9.3.1 Commonalities

- Agents act autonomously
- Skills map to MAS "capabilities"
- Orchestrators resemble coordination mechanisms

9.3.2 Differences

- MAS rarely includes **enterprise governance**
- AOW has **Assurance Levels, Policies, Guardian**, and **Provenance**
- AOW incorporates **Outcomes, Feedback, Memory** systematically

AOW adds enterprise trust, compliance, and business semantics missing from academic MAS literature.

9.4 AOW vs. Knowledge Graphs

AOW can be implemented atop a knowledge graph but is not equivalent to one.

- Knowledge graphs describe entities + relationships
- AOW describes the ontology of intelligent work and execution lifecycle

AOW is the semantic schema; a knowledge graph is one possible storage substrate.

10 Industry Example Set

To illustrate the extensibility of AOW, below are several domain-specific examples.

10.1 Insurance: Claims Adjudication

Objective: Reduce adjudication time

Intent: Automate eligibility triage

Context: Claimant age, policy details, historical exceptions

Policy: Auto-approve low-risk claims

Result: Eligibility decision

Outcome: SLA compliance improved

Feedback: Expand auto-approval criteria

10.2 Banking: KYC / AML Review

Objective: Improve compliance throughput

Intent: Automate document verification

Context: Customer risk profile, jurisdiction rules

Policy: High-risk cases require human review

Result: Verified or flagged documents

Outcome: Faster onboarding time

Feedback: Adjust anomaly thresholds

10.3 Healthcare: Prior Authorization

Objective: Reduce patient wait time

Intent: Validate medical necessity

Context: Clinical notes, insurance plan rules

Policy: Certain procedures require dual validation

Result: Authorization decision

Outcome: Treatment approval acceleration

Feedback: Update clinical decision guidelines

10.4 Public Sector: Benefits Determination

Objective: Reduce backlog

Intent: Automate eligibility checks

Context: Income data, household size

Policy: Sensitive cases escalate to specialists

Result: Eligibility decision

Outcome: Backlog reduced by X%

Feedback: Policy threshold tuning

10.5 Retail: Supply Chain Exception Handling

Objective: Reduce stockouts

Intent: Detect and resolve shipment anomalies

Context: Inventory levels, delays, routes

Policy: Auto-correct only if risk $\leq$ threshold

Result: Re-routing or replenishment

Outcome: Fewer lost sales

Feedback: Improve forecasting models

11 Validation Criteria

To evaluate the completeness and quality of AOW v3, we introduce validation criteria.

A mature ontology must have:

11.1 Semantic Completeness

- All necessary concepts (Intent, Agent, Policy, Outcome, etc.) are represented
- No entity is overloaded or ambiguous
- Each concept has a clear purpose and domain of meaning

11.2 Minimal Redundancy

- No two entities serve identical roles
- Entities like Result vs. Outcome, Memory vs. Knowledge Base, Assurance vs. Confidence are properly differentiated

11.3 Orthogonality

- Entities serve independent conceptual functions
- Example: Policies do not describe behavior, they constrain behavior

11.4 Extensibility

New entities (e.g., "Simulation", "Plan", "Worklet") can be added without breaking structure.

11.5 Formal Mapability

Ontology should be implementable using:

- JSON-LD
- RDF / OWL
- GraphQL schemas
- Knowledge graph frameworks

11.6 Governance-Ready

Ontology must express:

- Versioning
- Provenance
- Auditability
- Risk tiering
- Compliance constraints

11.7 Execution-Ready

Ontology should support:

- Dynamic planning
- Multi-agent cooperation
- Real-time context incorporation

11.8 Explainability-Ready

Mappings between:

- Intent → Action
- Action → Result
- Result → Outcome

must be reconstructable deterministically.

12 Glossary

Term	Definition
Objective	Business goal expressed in measurable terms
Intent	Structured, machine-interpretable goal
Context	Situational information that shapes Intent
Policy	Declarative constraints on behavior
Orchestrator	Component that decomposes Intent and assigns tasks
Agent	Actor capable of perceiving, reasoning, and acting
Skill	Atomic capability invoked by an Agent
Action	Execution event triggered by a Skill
Result	Immediate effect of an Action

Outcome	Business-level impact from aggregated Results
Confidence	Measure of certainty for a decision or action
Assurance Level	Required governance threshold
Feedback	Information used for improvement
Memory	Persistent execution knowledge
Operational Memory	Short-lived session or runtime state
Knowledge Base	Long-term learning repository
Guardian	Oversight mechanism for governance and policy
Provenance	Metadata that captures lineage
Versioning	Formal version control across entities

Appendix A: Formal Relationship Matrix

Below is a matrix of how each entity interacts with others.

Entity	Influences	Is influenced by
Objective	Intent	Outcome
Intent	Orchestrator, Agent	Objective, Context, Policy
Context	Intent	Memory
Policy	Agent, Intent	Guardian
Orchestrator	Agent	Intent, Policy
Agent	Skill, Result	Orchestrator, Policy
Skill	Result	Agent
Result	Outcome	Skill
Outcome	Feedback	Result
Feedback	Memory, Policy	Outcome

Memory	Context	Feedback
Guardian	Policy, Assurance	Governance model
Assurance Level	Autonomy	Risk model

Appendix B: Example Ontology Registry Format

A platform could store ontology entities in a registry like:

- ontology:
- entities:
- Objective
- Intent
- Context
- Policy
- Orchestrator
- Agent
- Skill
- Action
- Result
- Outcome
- Confidence
- AssuranceLevel
- Feedback
- Memory
- Guardian
- Provenance
- Versioning

13 Summary

AOW offers a **platform-agnostic, semantically rigorous, enterprise-ready ontology** that describes how intelligent work should be structured, governed, executed, and learned from.

It stands apart due to:

- Clear separation between Intent, Action, Outcome
- Strong governance constructs (Policy, Assurance, Guardian, Provenance)
- A full feedback-driven learning loop
- Rich relational semantics across agents, skills, and context
- Extensibility across industries

This ontology is positioned to become a foundational standard for Agentic Enterprise Automation in the next decade.